PROBABILITY & STATISTICS

for Machine Learning and Artificial Intelligence

A complete visual reference — from first principles to the probabilistic machinery behind neural networks, Bayesian reasoning, generative models, and modern evaluation. Every concept grounded in ML practice.

Part I: Foundations

1. **Probability Fundamentals** — Sample space, events, conditional probability, independence
2. **Bayes' Theorem** — Prior, likelihood, posterior — the engine of learning

Part II: Distributions

3. **Discrete Distributions** — Bernoulli, Binomial, Poisson — and where they appear in ML
4. **Continuous Distributions** — Gaussian, Beta, Exponential — the workhorses of probabilistic ML
5. **The Normal Distribution** — Anatomy, 68-95-99.7, why it appears everywhere

Part III: Summary Statistics

6. **Expectation and Variance** — Weighted averages, spread, Bias-Variance tradeoff
7. **Joint, Marginal & Conditional** — Multivariate probability, independence, correlation

Part IV: Information Theory

8. **Entropy, Cross-Entropy & KL** — Uncertainty, loss functions, distributional distance

Part V: Estimation

9. MLE and MAP — How loss functions are derived from probability
10. Bayesian Inference — Sequential updating, conjugate priors, posterior distributions

Part VI: Sampling & Testing

11. Central Limit Theorem — Why Gaussians appear in aggregates; standard errors
12. Bootstrapping — Model-free uncertainty estimation; used in Random Forests
13. Hypothesis Testing — p-values, Type I/II errors, A/B testing in ML

Part VII: ML-Specific Topics

14. Regression & Regularisation — OLS, Ridge, Lasso, overfitting through a probabilistic lens
15. Covariance, Correlation & PCA — Dimensionality reduction, feature decorrelation
16. Probabilistic Models — Naive Bayes, Gaussian Mixture Models, graphical models
17. Evaluation Metrics — Confusion matrix, ROC, AUC, precision-recall, F1
18. Monte Carlo Methods — Sampling-based integration, dropout as MC, MCMC intuition

Appendix: Quick Reference

A. Formula Cheat Sheet — All key formulas on one visual page

Part I · Chapter 1

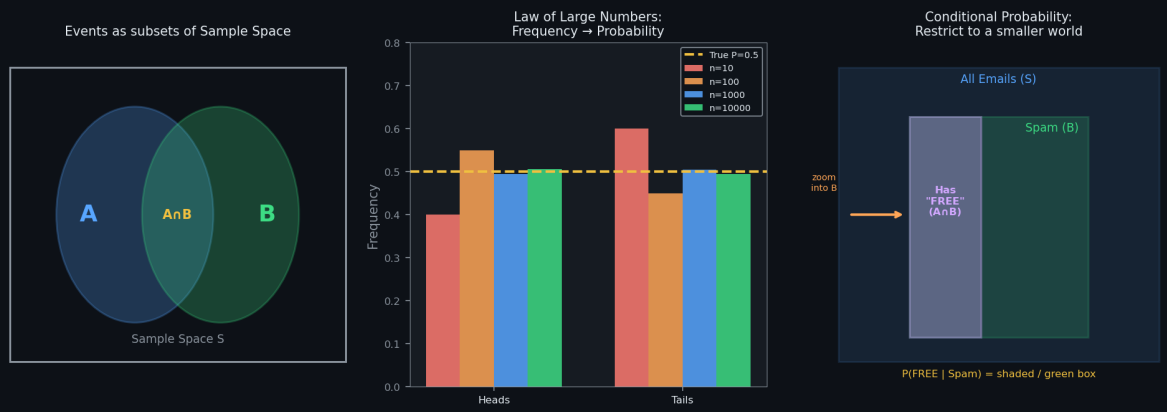
Probability Fundamentals

Probability is a language for quantifying uncertainty. Before a model makes a prediction, before a loss is computed, before a weight is updated — there is always a question being asked about what is more or less likely. This chapter builds the vocabulary every ML practitioner needs.

The Sample Space and Events

The **sample space S** is every possible outcome of an experiment. An **event** is any subset of the sample space — a collection of outcomes you care about. Probability assigns a number between 0 and 1 to each event, telling you how likely that collection of outcomes is.

> **Intuition:** Think of the sample space as a bag of all possible worlds. An event is a filter — it selects some worlds and excludes others. Probability is how large a fraction of worlds the filter keeps.



Left: Venn diagram of events A and B inside sample space S. A∩B (intersection) = both events occur. A∪B (union) = at least one occurs. Middle: Law of Large Numbers — frequency converges to probability. Right: Conditional probability as zooming into a restricted world.

Conditional Probability and Independence

P(A|B) — 'probability of A given B' — is the probability of A once you know B has happened. You effectively shrink the sample space to just the B-world and re-ask the question. Visually: zoom into the B circle and measure what fraction of it is also A.

Two events are **independent** if knowing one gives no information about the other: P(A|B) = P(A). This is the assumption that makes Naive Bayes tractable, and it is almost always wrong in the real world — which is why it is called 'naive.'

> **ML Connection — Conditional Probability**
> Every probabilistic classifier outputs P(class | features). Every attention weight is a conditional probability: P(attend to token j | query at position i). Language models predict P(next word | all previous words). Conditional probability is literally what neural networks learn to approximate.

Footnote: P(A|B) = P(A ∩ B) / P(B) | Chain rule: P(A,B) = P(A|B)·P(B)

The Three Axioms (Kolmogorov)

- $P(A) >= 0$ for all events A
- $P(S) = 1$ (something always happens)
- $P(A \cup B) = P(A) + P(B)$ if A and B are mutually exclusive

Everything else — Bayes' theorem, conditional probability, independence — is derived from these three rules. They are the bedrock. Once you accept them, the rest is logic.

Complement rule: $P(A^c) = 1 - P(A)$. Addition rule: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.

Part I · Chapter 2

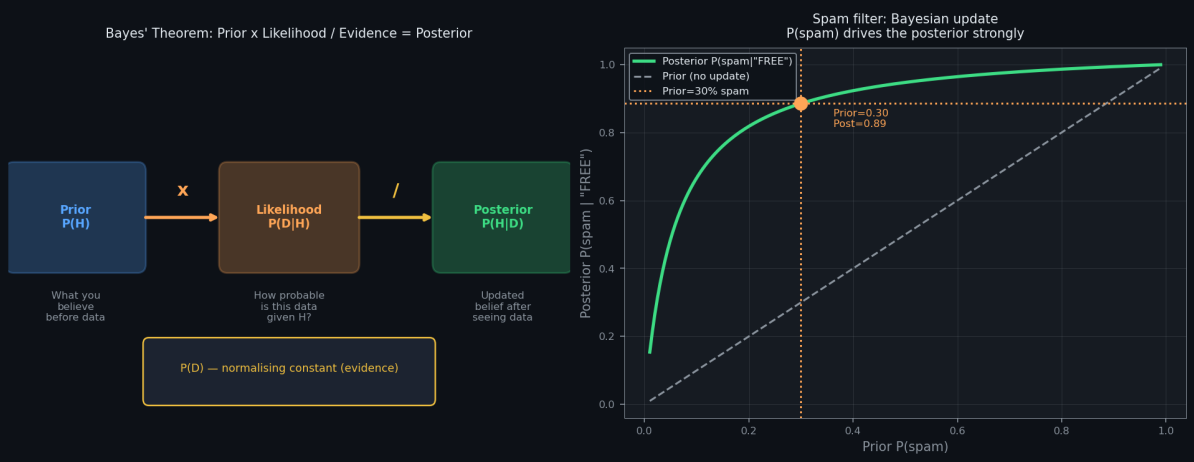
Bayes' Theorem

Bayes' theorem is not just a formula — it is a philosophy of learning. Start with a belief (prior). Observe data. Update. This update rule is the mathematical foundation of Bayesian neural networks, probabilistic graphical models, and much of modern machine learning.

The Update Equation

You have a hypothesis H. You observe data D. How should your belief in H change? Bayes tells you: multiply your prior belief by how likely the data is under that hypothesis (likelihood), then normalise so everything sums to 1.

> **Intuition:** You suspect a coin is biased. Your prior: probably fair. You flip it 10 times: 9 heads. The likelihood of that under a fair coin is tiny. Bayes forces your posterior toward 'biased.' Data overrules gut feeling — proportionally.



Left: Bayes' theorem as a three-stage pipeline — prior × likelihood / evidence = posterior. Right: How a Bayesian spam filter's posterior changes as the prior probability of spam varies. Base rate matters enormously — a common cognitive error known as base rate neglect.

Why This Is the Basis of ML Loss Functions

Maximum Likelihood Estimation (MLE) is just Bayes with a flat prior. MAP estimation is Bayes with an informative prior. L2 regularisation (Ridge) corresponds to a Gaussian prior on weights. L1 regularisation (Lasso) corresponds to a Laplace prior. The entire zoo of loss functions is Bayesian inference in disguise.

> **ML Connection — Naive Bayes Classifier**
> A Naive Bayes spam filter computes P(spam | words) using Bayes' theorem. P(spam) = prior (fraction of emails that are spam). P(words | spam) = likelihood (how often these words appear in spam). P(words) = normaliser. Despite the 'naive' independence assumption, this classifier often outperforms far more complex models when data is scarce.

Footnote: $P(H|D) = P(D|H) \cdot P(H) / P(D)$ where $P(D) = \Sigma P(D|H_i) \cdot P(H_i)$ (law of total probability)

Part II · Chapter 3

Discrete Distributions

A probability distribution describes the complete picture of how probability is spread across outcomes. Discrete distributions handle countable outcomes: yes/no, counts, integers. Knowing which distribution to reach for in which situation is a core ML modelling skill.

Bernoulli — The Coin Flip

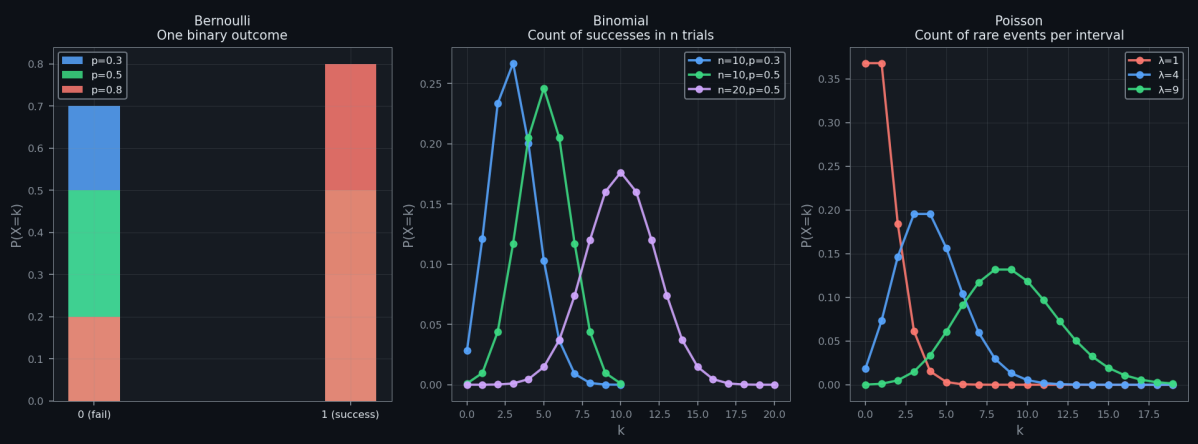
The simplest distribution. One trial. Two outcomes: success (1) or failure (0). One parameter: p (probability of success). Every binary classification model's output — spam/not-spam, cat/dog, click/no-click — is a Bernoulli random variable. The binary cross-entropy loss is derived from the Bernoulli likelihood.

Binomial — Counting Successes

Run n independent Bernoulli trials. Count the successes. That count follows a Binomial distribution. In ML: the number of correct predictions in a batch, the number of relevant documents retrieved — any 'how many times did X happen in N chances?' question is Binomial.

Poisson — Rare Events per Interval

When events happen independently and rarely, and you count how many occur in a fixed window, you get a Poisson distribution. In ML: number of errors per 1000 tokens generated, number of anomalies per hour, event counts in time-series models.



Left: Bernoulli PMF for different values of p. Middle: Binomial PMF — note how it becomes bell-shaped as n grows (CLT in action). Right: Poisson PMF — heavier right tail, controlled by λ (the mean).

ML Connection — Where Discrete Distributions Appear

Bernoulli: binary classification output, dropout (each neuron dropped with Bernoulli(p)). Binomial: evaluating models on fixed test sets, confidence intervals on accuracy. Categorical (generalised Bernoulli): softmax output is a Categorical distribution. Poisson: count-based NLP tasks (number of words, number of edits), anomaly detection, modelling token frequency.

PMF = Probability Mass Function. For discrete X: P(X=k). Bernoulli: P(X=1)=p. Binomial: C(n,k)·p^k·(1-p)^(n-k). Poisson: e^(-λ)·λ^k/k!

Part II · Chapter 4

Continuous Distributions

When outcomes are real-valued — weights, activations, measurements, probabilities — you need continuous distributions. Probability is no longer assigned to exact values but to ranges (intervals), as areas under density curves.

Gaussian — The Universal Distribution

The Normal (Gaussian) distribution is defined by just two parameters: mean (mu, the centre) and standard deviation (sigma, the spread). It appears everywhere because of the Central Limit Theorem: sums of many independent random variables converge to Gaussian, regardless of the original distribution.

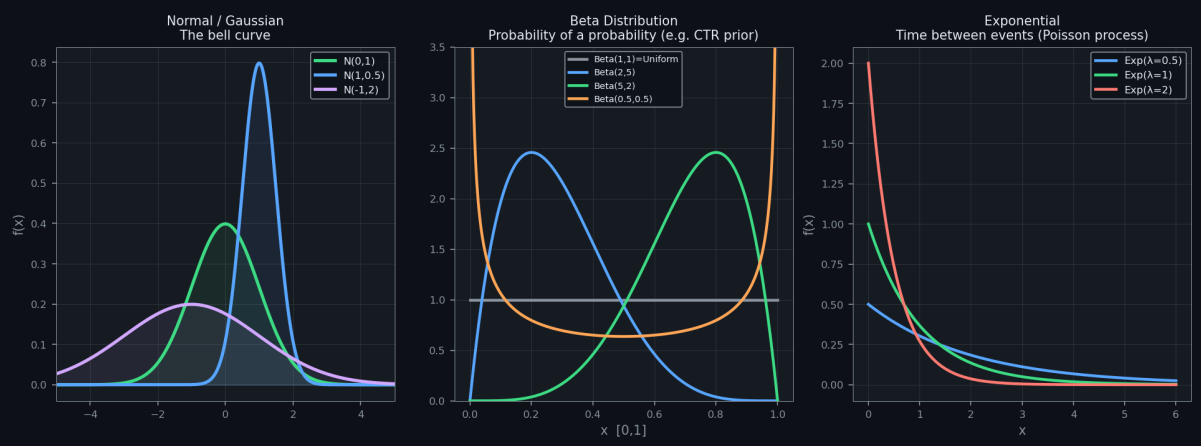
In ML: weight initialisation, noise models, variational autoencoders (latent space), Gaussian processes, Bayesian neural networks.

Beta — Distribution Over Probabilities

The Beta distribution lives on [0,1], making it perfect for modelling probabilities themselves — a probability of a probability. Its two shape parameters (alpha, beta) control whether it concentrates near 0, near 1, near the middle, or at the extremes. In ML: Bayesian click-through-rate estimation, prior for proportion parameters, policy in reinforcement learning.

Exponential — Time Between Events

The Exponential distribution models the waiting time between events in a Poisson process. It is memoryless: past waiting time gives no information about future waiting time. In ML: survival analysis, time-series modelling, event-driven reinforcement learning.



Left: Gaussian family — shift mu to move, change sigma to stretch or compress. Middle: Beta family — a flexible distribution over [0,1], the natural conjugate prior for Bernoulli/Binomial likelihoods. Right: Exponential family — heavier or lighter tail controlled by rate λ.

PDF = Probability Density Function. $P(a < X < b)$ = integral from a to b of $f(x)dx$. Gaussian: $f(x) = (1/\sigma\sqrt{2\pi})\cdot\exp(-(x-\mu)^2/2\sigma^2)$. Beta: $f(x) = x^{(\alpha-1)}\cdot(1-x)^{(\beta-1)}/B(\alpha,\beta)$. Exp: $f(x) = \lambda\cdot e^{(-\lambda x)}$.

Part II · Chapter 5

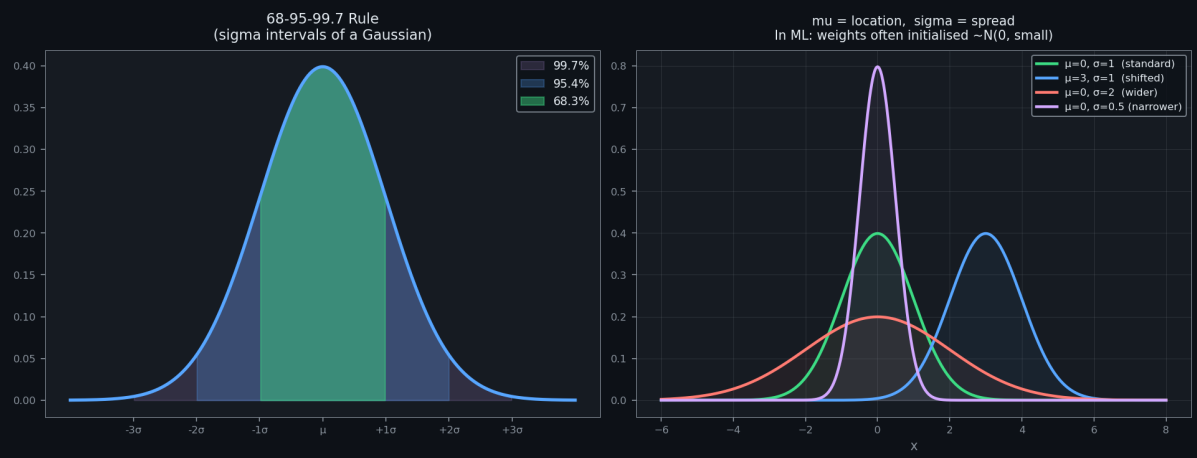
Anatomy of the Normal Distribution

The Gaussian deserves its own chapter. It is the most important distribution in statistics and machine learning. Understanding its shape, its parameters, and the 68-95-99.7 rule gives you immediate practical power for reading model outputs, confidence intervals, and initialisation strategies.

The 68-95-99.7 Rule

For any Gaussian, exactly 68.3% of values fall within 1 sigma of the mean, 95.4% within 2 sigma, and 99.7% within 3 sigma. This is not an approximation — it is exact. Memorise these three numbers. They are used constantly.

> **Intuition:** If a model's weight is initialised from N(0, 0.01), then 99.7% of weights will start between -0.03 and +0.03. If a z-score on your test error is 2.3, the result is outside 2 sigma — happening less than 5% of the time by chance.



Left: 68-95-99.7 rule visualised. The tightest (green) band holds 68%. Right: how mu shifts the centre and sigma controls the width. Narrower = more concentrated = more confident.

Standard Normal and Z-Scores

The **standard normal Z ~ N(0,1)** has mean 0 and standard deviation 1. Any Gaussian can be converted to it: Z = (X - mu) / sigma. This z-score tells you how many standard deviations X is from the mean. Z-scores are used in batch normalisation, layer normalisation, and standardisation of features before training.

ML Connection — Batch Normalisation

Batch norm computes the z-score of each activation: $z = (x - \mu_batch) / \sigma_batch$, then rescales: $y = \gamma z + \beta$ (learnable γ, β). This stabilises training by ensuring activations stay in a sensible range. It is literally standardising to a near-Gaussian at each layer. Layer norm (used in Transformers) applies the same idea across features rather than batch.

CDF of standard normal: $\Phi(z) = P(Z \leq z)$. No closed form — look up or use scipy.stats.norm.cdf. Softmax output is a Categorical, but the logits before softmax are often assumed Gaussian.

Part III · Chapter 6

Expectation, Variance and Bias-Variance Tradeoff

Two numbers capture most of what matters about a distribution: where it is centred (expectation) and how spread out it is (variance). In ML, these concepts generalise into the famous bias-variance tradeoff — the fundamental reason why no model is perfect.

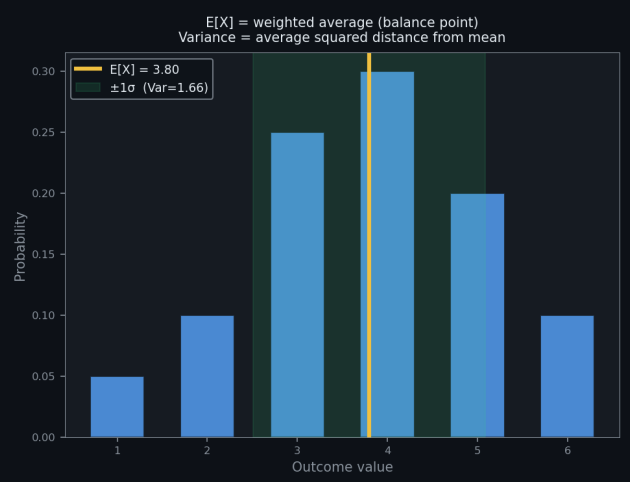
Expectation — The Weighted Centre of Gravity

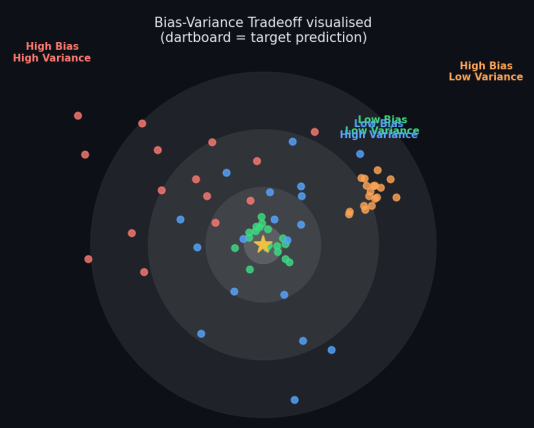
E[X] is the average value of X if you repeated the experiment infinitely many times. It is the balance point of the probability distribution — not necessarily the most likely value, but the average value. For a fair die: E[X] = 3.5, even though 3.5 never appears.

Key properties: E[aX+b] = a·E[X]+b (linearity). E[X+Y] = E[X]+E[Y] (always, even if X,Y are dependent). These properties make expectation easy to reason with.

Variance — The Average Squared Deviation

Var[X] measures how spread out X is around its mean. It is the average of the squared distances from E[X]. Squaring ensures positive values and penalises large deviations more than small ones. Standard deviation sigma = sqrt(Var[X]) brings it back to the original units.





Left: expectation as the balance point of a discrete distribution, with variance shown as a symmetric band. Right: the bias-variance tradeoff visualised as a dartboard. The goal is low bias AND low variance — the top-left quadrant.

Bias-Variance Tradeoff — The Core Tension in ML

For any ML model, generalisation error = Bias² + Variance + Irreducible Noise.

Bias: systematic error — the model is wrong in a consistent direction (underfitting, too simple). **Variance**: sensitivity to data — the model changes wildly with different training sets (overfitting, too

complex).

There is an unavoidable tradeoff. More capacity reduces bias but increases variance. Regularisation adds bias back in to control variance. More data reduces variance without increasing bias — which is why scale is such a powerful force in modern ML.

ML Connection — Bias-Variance in Practice

High bias: training loss ≈ validation loss, both high. Add capacity or features. High variance: training loss much lower than validation loss. Add data or regularise. Ensemble methods (Random Forests, Boosting) explicitly target the bias-variance tradeoff: bagging (averaging many models) reduces variance; boosting (correcting errors sequentially) reduces bias.

$Var[aX] = a^2 \cdot Var[X]$. $Var[X+Y] = Var[X]+Var[Y]$ only if X,Y independent. Std dev is more interpretable than variance (same units as X).

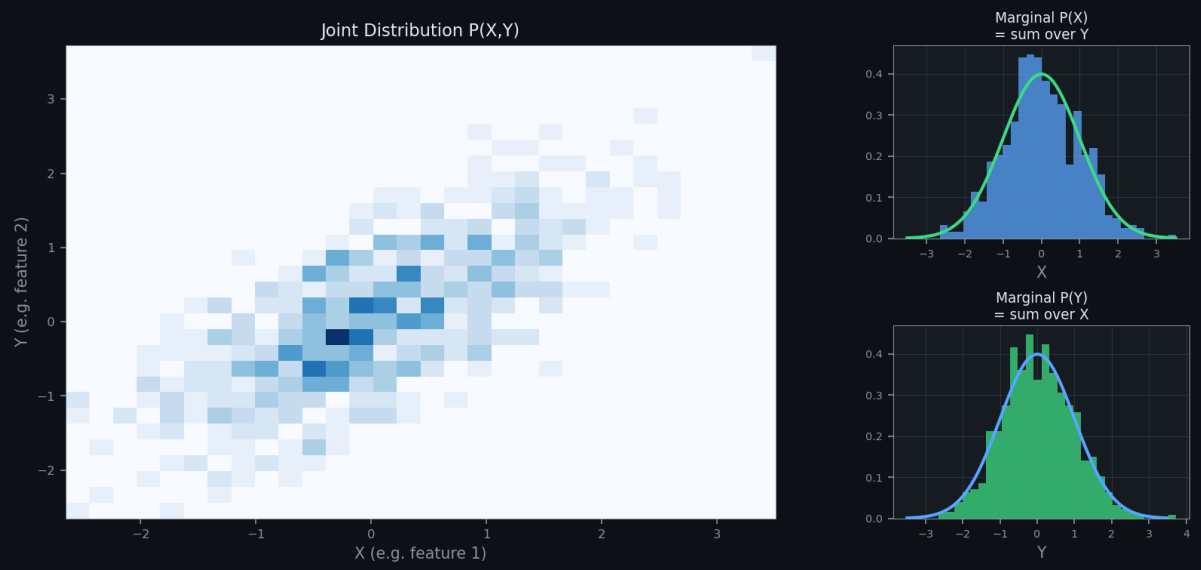
Part III · Chapter 7

Joint, Marginal and Conditional Distributions

Real data has many features that are not independent. Joint distributions describe all variables together. Marginal distributions describe one variable in isolation. Understanding the relationships between variables is what feature engineering, dimensionality reduction, and causal inference are built on.

Joint and Marginal Distributions

The **joint distribution P(X,Y)** gives the probability of every combination of X and Y. It contains complete information about both variables and their relationship. The **marginal distribution P(X)** is obtained by 'summing out' Y — you integrate over all possible Y values to see X alone.

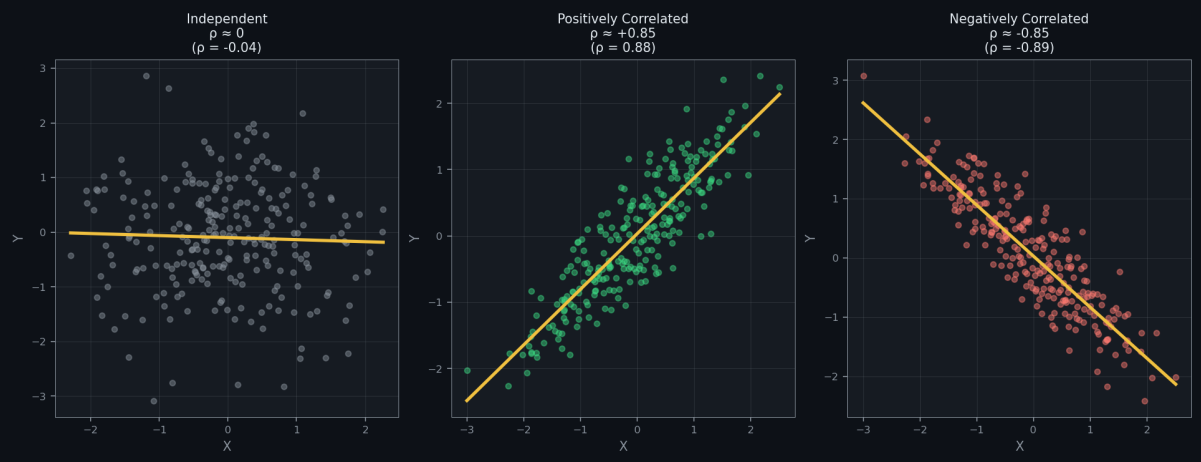


The 2D histogram (left) shows the joint distribution of two correlated features. The side panels show the marginal distributions — what each feature looks like in isolation, ignoring the other.

Correlation and Independence

Correlation (rho) measures linear dependence between two variables, ranging from -1 to +1. rho=0 means no linear relationship — but does NOT mean independence. Independence means P(X,Y) = P(X)·P(Y) — knowing X tells you absolutely nothing about Y. Independence implies rho=0, but rho=0 does NOT imply independence.

> **Watch out:** Correlation measures only linear relationships. Two variables can have rho=0 and still be perfectly dependent (e.g. $Y = X^2$). In ML: mutual information captures non-linear dependence; always visualise your data before trusting a correlation coefficient.



Three scatter plots with regression lines. Independence (left), positive correlation (middle), negative correlation (right). The regression line slope reflects rho and the ratio of standard deviations.

$Cov[X,Y] = E[(X-\mu x)(Y-\mu y)] = E[XY] - E[X]\cdot E[Y]$. $\rho = Cov[X,Y]/(\sigma x\cdot\sigma y)$. Independence: $P(X,Y)=P(X)\cdot P(Y)$.

Part IV · Chapter 8

Entropy, Cross-Entropy and KL Divergence

Information theory gives us precise tools for measuring uncertainty, comparing distributions, and deriving loss functions. Cross-entropy loss, KL divergence in VAEs, mutual information in representation learning — all come from this one framework.

Entropy — Measuring Uncertainty

Entropy H(X) is the average amount of information (surprise) in a distribution. A deterministic outcome (probability 1 for one option) has entropy 0 — no surprise. A uniform distribution over N outcomes has maximum entropy log(N) — maximum uncertainty. Entropy is measured in bits (if using log base 2) or nats (natural log).

> **Intuition:** Entropy is also the minimum number of bits needed on average to encode outcomes. A biased coin needs fewer bits per toss than a fair coin — because the outcome is more predictable.

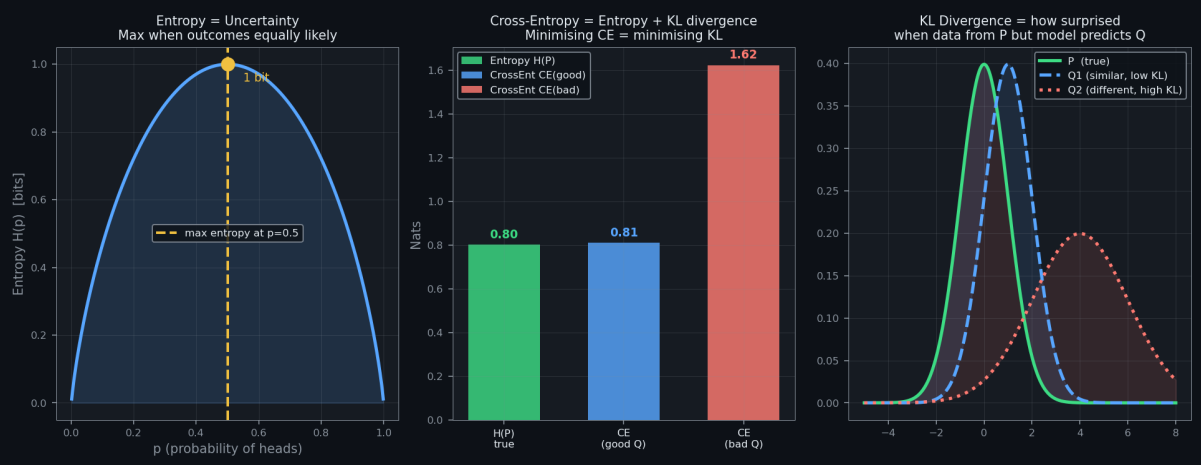
Cross-Entropy — The Classification Loss

Cross-entropy CE(P, Q) measures the average surprise when the true distribution is P but your model predicts Q. When Q = P: CE = H(P) (minimum possible). When Q is wrong: CE > H(P). The excess is KL divergence.

This is why **minimising cross-entropy loss = making your model's distribution as close as possible to the true data distribution**. It is the most principled loss function for classification problems.

KL Divergence — Distributional Distance

KL(P||Q) measures how much extra surprise you get when using Q instead of P. It is always >= 0 (Gibbs' inequality). It is NOT symmetric: KL(P||Q) != KL(Q||P). In VAEs: the KL term forces the latent space toward a standard Gaussian prior. In RL: KL prevents the policy from changing too drastically in one update (PPO, TRPO).



Left: Entropy of a coin flip — maximum at p=0.5 (1 bit). Middle: Cross-entropy is larger than entropy by exactly KL divergence — so minimising CE drives the model Q toward the true P. Right: KL divergence as the area of mismatch between two distributions.

ML Connection — Where These Appear

Cross-entropy loss: classification (binary and multi-class). KL divergence: VAE loss = reconstruction loss + KL(posterior || prior). Mutual information I(X;Y) = KL(P(X,Y) || P(X)P(Y)): used in ICA, InfoNCE (contrastive learning). Entropy regularisation in RL (SAC): maximise reward + entropy to encourage exploration. Perplexity of a language model = exp(cross-entropy) — standard LLM evaluation metric.

$H(X) = -\Sigma P(x) \cdot \log P(x)$. $CE(P,Q) = -\Sigma P(x) \cdot \log Q(x)$. $KL(P||Q) = \Sigma P(x) \cdot \log(P(x)/Q(x)) = CE(P,Q) - H(P)$. Jensen-Shannon divergence JSD = symmetric, bounded version of KL.

Part V · Chapter 9

Maximum Likelihood and MAP Estimation

How do you choose model parameters given data? MLE and MAP are the two fundamental answers. Together they explain where every standard ML loss function comes from and what regularisation is doing probabilistically.

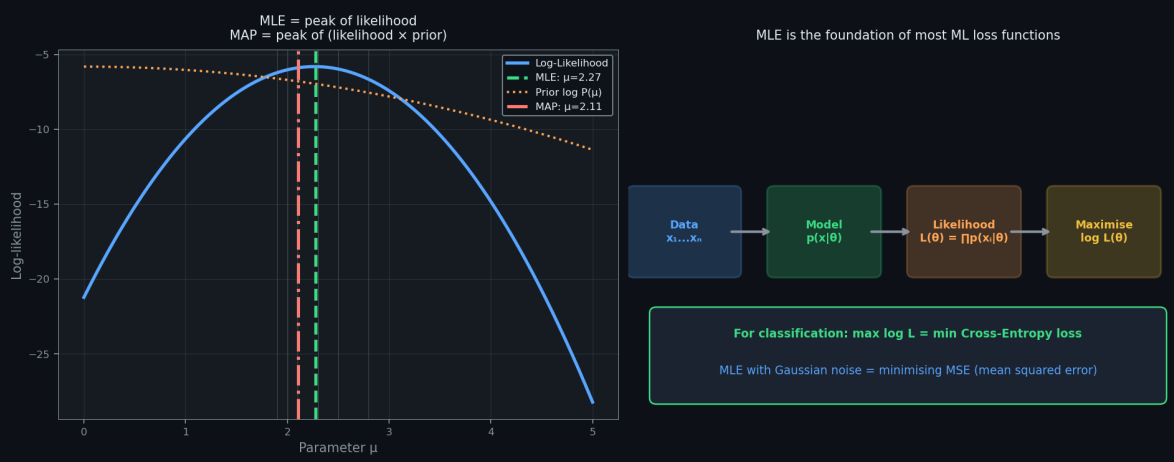
MLE — Parameters That Make Data Most Probable

MLE asks: for what parameter values theta is the observed data most probable? You write down the probability of all data points as a function of theta, then maximise it. Taking logs converts products to sums and does not change the argmax — log-likelihood is easier to work with.

> **Intuition:** Imagine you flip a coin 10 times and get 7 heads. MLE says: what value of p makes 7 heads in 10 flips most likely? The answer is p=0.7 — because that directly maximises the Binomial likelihood. MLE always picks the parameter that makes the data look least surprising.

MAP — Adding a Prior to MLE

MAP (Maximum A Posteriori) adds a prior P(theta) to MLE. Instead of maximising just the likelihood, you maximise likelihood × prior. The prior acts as a soft constraint, pulling the estimate toward what you believed before seeing data. With a flat prior, MAP = MLE.



Left: the log-likelihood landscape for a mean parameter. MLE finds the peak of the likelihood (green). MAP is pulled toward the prior's peak (red) — the balance depends on data size. Right: the chain from data to MLE, and its equivalence to minimising standard losses.

ML Connection — Loss Functions as Probabilistic Inference

MSE loss = MLE assuming Gaussian noise: minimising ||y - ■||² is equivalent to maximising the Gaussian likelihood. Binary cross-entropy = MLE for Bernoulli outputs. Categorical cross-entropy = MLE for softmax outputs. L2 regularisation (Ridge) = MAP with Gaussian prior on weights. L1 regularisation (Lasso) = MAP with Laplace prior on weights. Weight decay in Adam = L2 regularisation = Gaussian prior.

log L(θ) = Σ log P(x■|θ). MAP: θ = argmax [log L(θ) + log P(θ)]. Regularised loss = -log L(θ) - log P(θ).*

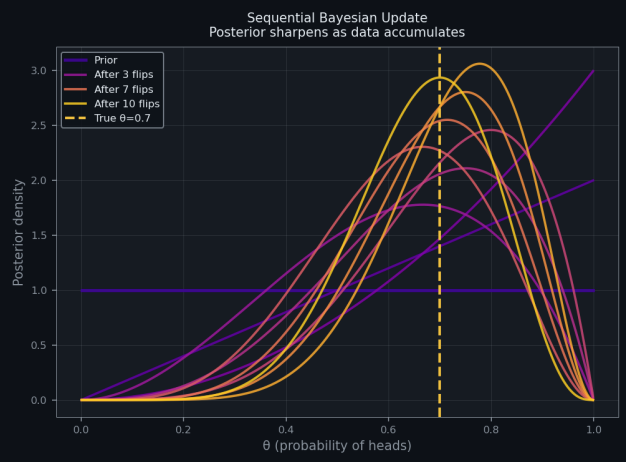
Part V · Chapter 10

Bayesian Inference

Full Bayesian inference does not just find a single best parameter — it maintains a complete distribution over all possible parameters. This gives you calibrated uncertainty estimates, not just point predictions. This is the foundation of Bayesian deep learning and probabilistic ML.

The Full Posterior

Rather than asking 'what is the best theta?', Bayesian inference asks 'what is the distribution over theta given the data?' This posterior distribution captures everything the data has told you about the parameters, including uncertainty. A wide posterior means high parameter uncertainty. A narrow posterior means the data has pinned down the parameter well.



Conjugate Priors: Choose prior to match likelihood

Likelihood	Prior	Posterior	ML Use Case
Bernoulli	Beta	Beta	Click-through rate
Gaussian (μ)	Gaussian	Gaussian	Bayesian regression
Gaussian (σ^2)	Inverse-Gamma	Inverse-Gamma	Uncertainty estimate
Poisson	Gamma	Gamma	Event rate modelling
Multinomial	Dirichlet	Dirichlet	Topic models, LDA

Left: Sequential Bayesian updating for a coin's bias. The posterior starts as a flat prior and concentrates around the true value (0.7) as more data arrives. Right: conjugate prior table — choosing priors that have closed-form posteriors.

Conjugate Priors — Closed-Form Posteriors

For certain combinations of likelihood and prior, the posterior has the same family as the prior — just with updated parameters. This is called a conjugate prior. No integration needed. The update is just arithmetic on the parameters. This is why Beta-Bernoulli and Gaussian-Gaussian are so common in practice.

ML Connection — Bayesian Methods in Practice

Bayesian hyperparameter optimisation (Gaussian Process surrogate) maintains a distribution over which hyperparameters are best, guiding search efficiently. Bayesian neural networks (BNNs) put distributions over weights, enabling uncertainty quantification — critical for safety-critical AI. Thompson sampling in recommendation systems uses Bayesian posteriors to balance exploration vs exploitation. MC Dropout approximates Bayesian inference cheaply at test time.

Part VI · Chapter 11

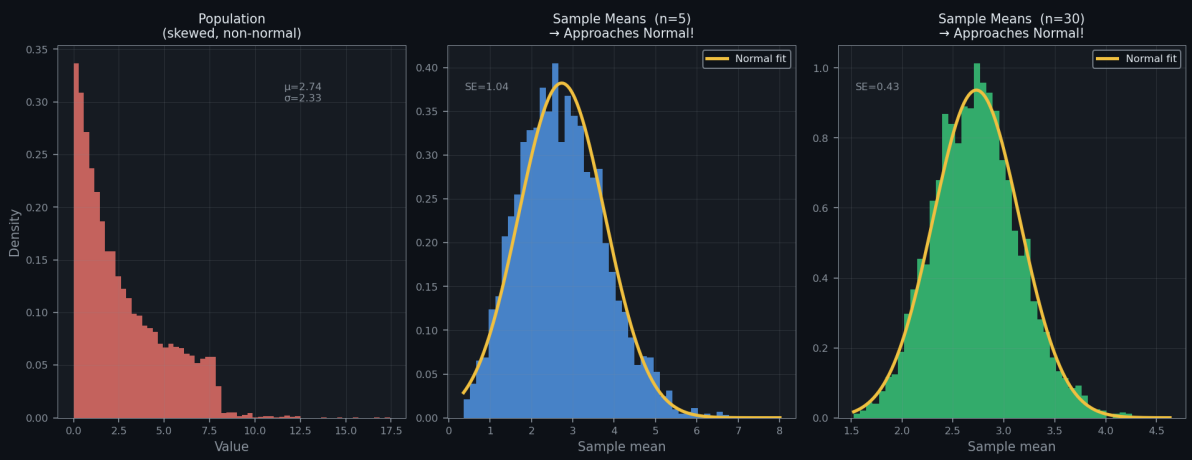
The Central Limit Theorem

The Central Limit Theorem (CLT) is perhaps the most astonishing fact in all of statistics: add up enough independent random variables and, regardless of their individual distributions, the sum is approximately Gaussian. This is why Gaussian distributions appear everywhere in nature and ML.

The Theorem

Take any distribution with finite mean mu and variance sigma². Draw samples of size n. Compute the sample mean. As n grows, the distribution of sample means converges to N(mu, sigma²/n). The convergence is fast — often n=30 is already quite good.

> **Intuition:** The CLT is why we can use t-tests and z-tests even when the underlying data is not Gaussian. It is why training loss averaged over a mini-batch behaves nicely. It is why ensembles of many weak predictors become reliable.



A highly skewed, non-Gaussian population (left). Sampling distributions of the mean for n=5 (middle) and n=30 (right). Even a sample size of 30 from a very skewed distribution produces a near-perfect Gaussian of sample means.

> **ML Connection — CLT in Training**
>
> Mini-batch gradient descent computes the mean gradient over a batch. By CLT, this batch mean gradient is approximately Gaussian around the true gradient. Larger batch size → smaller variance of gradient estimate → more stable training. Standard errors in ML benchmarking: when you run k experiments, the mean performance ± 1.96·(std/√k) gives a 95% confidence interval (CLT). The Gaussian noise assumed in diffusion models is motivated by the CLT.

Standard Error (SE) = σ/√n = std dev of the sampling distribution of the mean. 95% CI for mean: x̄ ± 1.96·SE. CLT requires finite variance — heavy-tailed distributions (Cauchy) are exceptions.

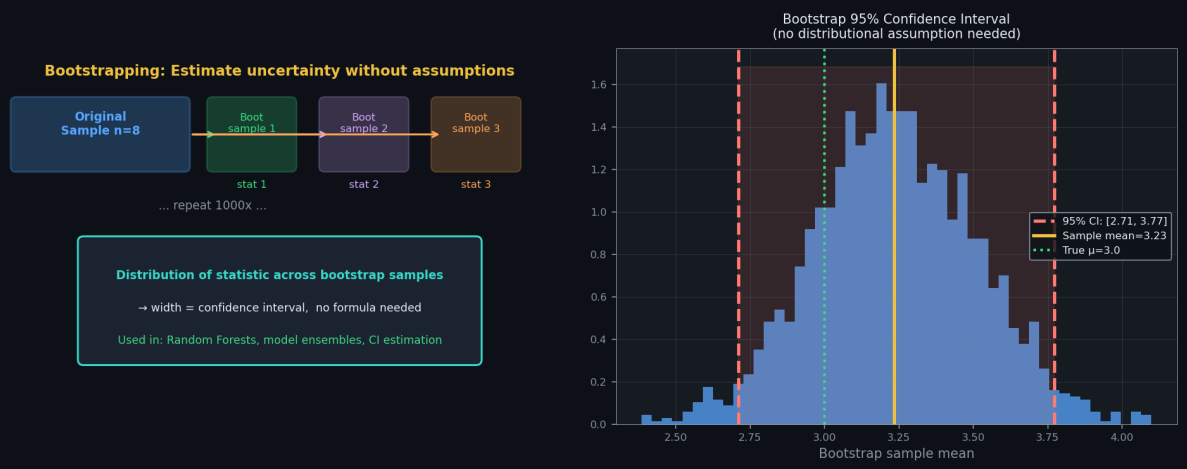
Part VI · Chapter 12

Bootstrapping

Bootstrapping is a computational technique for estimating uncertainty without assuming any distributional form. It works by treating the sample itself as a proxy for the population and repeatedly resampling from it to simulate the sampling process.

The Core Idea

You have a sample. You want to know how uncertain your estimate is. Bootstrap: resample your data with replacement (same size, but duplicates allowed). Compute your statistic on each resample. Repeat 1000 times. The spread of these 1000 estimates tells you your uncertainty.



Left: the bootstrapping workflow. Each resample is drawn with replacement from the original. The distribution of statistics across resamples gives you a confidence interval — no distributional assumptions needed. Right: a bootstrap 95% CI for the mean, showing the true value is captured.

ML Connection — Bootstrapping in Ensemble Methods

Random Forests train each tree on a bootstrap resample of the training data (this is called 'bagging' — Bootstrap AGGregatING). The variance reduction from averaging predictions across trees is exactly the variance reduction from averaging bootstrap estimates. Bootstrapping is also used to compute confidence intervals on model accuracy, AUC, F1 score, and other metrics when the test set is small.

Bootstrap percentile CI: sort the B bootstrap statistics, take the 2.5th and 97.5th percentiles. BCa (bias-corrected accelerated) bootstrap is more accurate for skewed distributions.

Part VI · Chapter 13

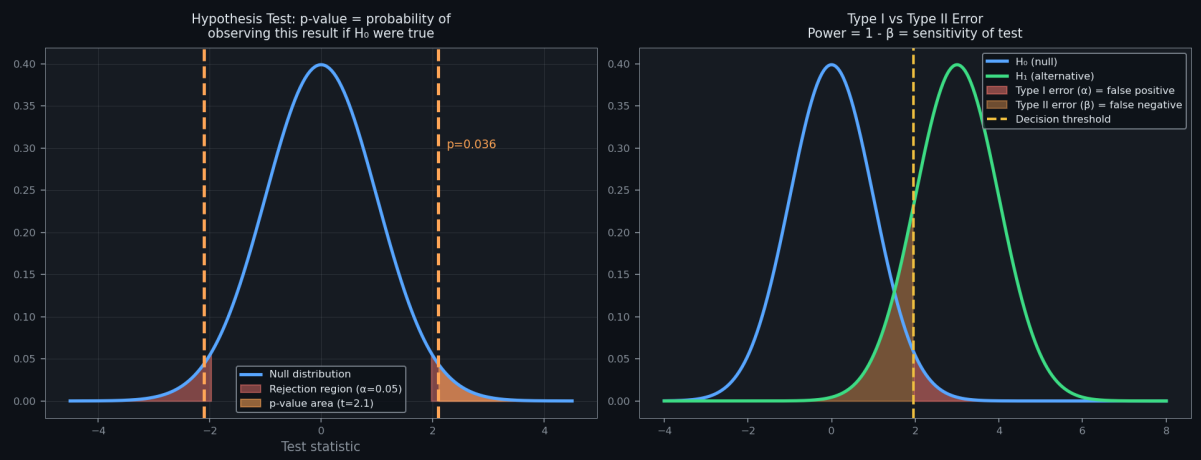
Hypothesis Testing

Hypothesis testing formalises the question: 'Could this result have happened by chance?' In ML, this means A/B testing, comparing model variants, checking if an improvement is statistically significant, and understanding the dangers of multiple comparisons.

The Framework

State a null hypothesis H0 (e.g. 'model A and B have equal accuracy'). Compute a test statistic from your data. Find the p-value: the probability of seeing a result this extreme or more extreme, if H0 were true. If p < alpha (usually 0.05), reject H0.

> **Intuition:** The p-value is NOT the probability that H0 is true. It is the probability of your data, assuming H0 is true. These are completely different things — and confusing them is one of the most common errors in ML papers.



Left: the null distribution (what you'd see by chance) with the rejection region shaded red. The p-value is the tail area beyond the observed test statistic. Right: Type I error (false positive) and Type II error (false negative) shown as overlapping null and alternative distributions.

Type I and Type II Errors — The Tradeoff

Type I error (alpha): rejecting H0 when it is actually true. A false positive. You conclude the new model is better when it is not.

Type II error (beta): failing to reject H0 when it is actually false. A false negative. You miss a real improvement.

Power = 1 - beta: the probability of detecting a real effect when it exists. Higher power requires larger sample size or larger true effect.

Watch out: Multiple comparisons problem (p-hacking): if you test 20 hypotheses at alpha=0.05, you expect one false positive by chance. In ML: running many experiments and reporting only the best result inflates false positives. Use Bonferroni correction or control FDR (Benjamini-Hochberg).

ML Connection — A/B Testing for Models

Comparing two model variants: compute the metric difference on a held-out test set. Use a paired t-test or Wilcoxon signed-rank test to get a p-value. Rule of thumb: a 95% CI that does not cross 0 → statistically significant. McNemar's test for comparing classifiers on the same test set. Always report effect size (Cohen's d) alongside p-value — a tiny, meaningless improvement can be statistically significant with a huge test set.

Part VII · Chapter 14

Regression, Regularisation and Overfitting

Linear regression is not just a simple model — it is the analytical foundation for understanding how any model fits data. Regularisation, overfitting, and the bias-variance tradeoff all become concrete and visible in the regression setting.

Ordinary Least Squares — Minimising Residuals

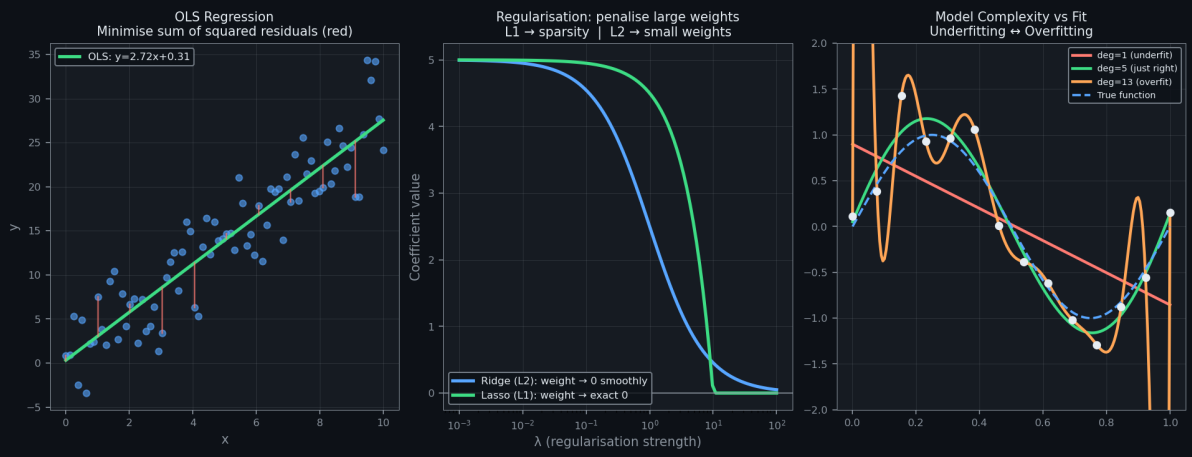
OLS finds the line that minimises the sum of squared vertical distances (residuals) between predictions and data. This is identical to MLE assuming Gaussian noise: the two derivations give the same answer.

Regularisation — The Probabilistic View

Ridge (L2): adds lambda·||w||² to the loss. Probabilistically: Gaussian prior on weights. Effect: all weights shrink toward 0, none become exactly 0. Good for multicollinearity.

Lasso (L1): adds lambda·||w||■ to the loss. Probabilistically: Laplace prior. Effect: weights are pushed to exactly 0 — automatic feature selection. Produces sparse models.

Elastic Net: combines L1 and L2. Used in deep learning as a combination of weight decay and sparsity.



Left: OLS regression with residuals (red lines). Middle: how Ridge and Lasso differ — L1 hits zero exactly, L2 approaches asymptotically. Right: polynomial fit — degree-1 underfits, degree-5 fits well, degree-13 overfits wildly.

ML Connection — Regularisation in Deep Learning

Weight decay in Adam is exactly L2 regularisation (Gaussian prior on weights). Dropout can be interpreted as L2 regularisation with a noisy approximation. Data augmentation reduces variance (effective increase in n). Early stopping is implicit regularisation — you stop before the model memorises noise. Batch norm also has a regularising effect by adding noise to activations.

OLS solution: w = (X^T X)^(-1) X^T y. Ridge: w = (X^T X + λI)^(-1) X^T y. Lasso: no closed form — requires iterative solver (coordinate descent).

Part VII · Chapter 15

Covariance, Correlation and PCA

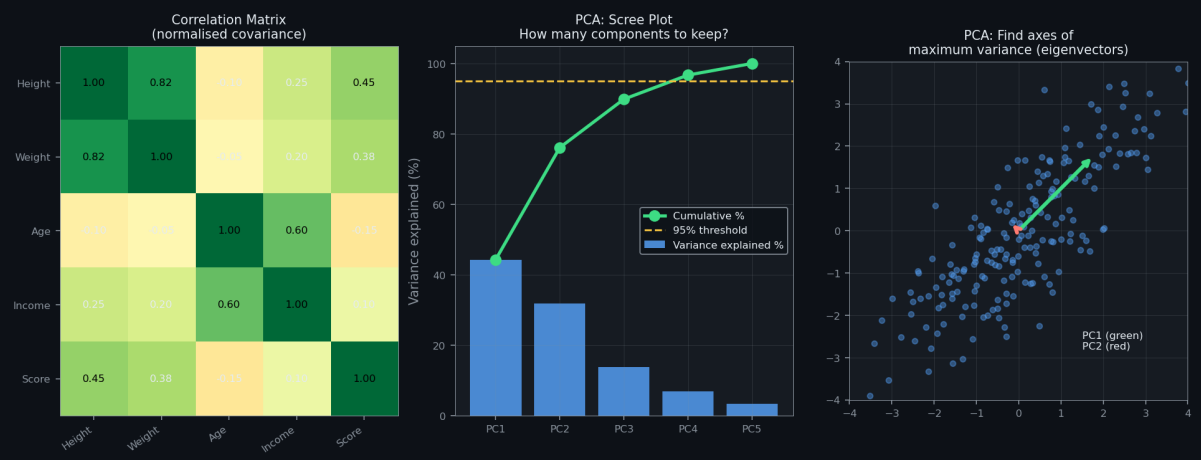
Real datasets have many features that are correlated. Understanding and exploiting this structure is the purpose of covariance matrices, PCA, and linear dimensionality reduction. These are also the building blocks of Gaussian processes and LDA.

Covariance Matrix — The Geometry of Features

For n features, the covariance matrix is n×n. Entry (i,j) is Cov[Xi, Xj] — how features i and j move together. The diagonal entries are variances. The off-diagonal entries capture linear relationships. Normalising by standard deviations gives the correlation matrix with values in [-1,1].

PCA — Finding the Axes of Maximum Variance

PCA finds a new coordinate system where the axes (principal components) point in the directions of maximum variance. Mathematically: compute eigenvectors of the covariance matrix. The eigenvector with the largest eigenvalue is PC1 — the direction of maximum spread. Projecting data onto the top k PCs reduces dimensionality while preserving as much variance (information) as possible.



Left: correlation matrix heatmap of 5 features. Middle: scree plot — variance explained by each principal component. The 'elbow' suggests how many components to keep. Right: eigenvectors of a 2D covariance matrix, showing the principal axes of the data cloud.

ML Connection — Covariance Everywhere

Weight matrices in neural networks can be analysed through their singular value decomposition (closely related to PCA). Attention in Transformers computes a scaled dot-product — which is proportional to the sample covariance of keys and queries. Gaussian Processes use covariance kernels (RBF, Matern) to define similarity. PCA is used for visualisation (t-SNE and UMAP both initialise from PCA), denoising, and whitening data before training.

Covariance matrix $\Sigma = E[(X-\mu)(X-\mu)^T]$. Eigendecomposition: $\Sigma = V\Lambda V^T$ where columns of V are eigenvectors. PCA: project data onto top k eigenvectors. Variance explained = $\lambda_k/\Sigma\lambda$.

Part VII · Chapter 16

Probabilistic Models — Naive Bayes and GMMs

Probabilistic models make their assumptions explicit. They are interpretable, uncertainty-aware, and data-efficient. Naive Bayes and Gaussian Mixture Models are two of the most widely used probabilistic ML models, each making clear geometric and statistical assumptions.

Naive Bayes Classifier

Naive Bayes applies Bayes' theorem with the 'naive' assumption that all features are conditionally independent given the class. Despite this often being false, it works surprisingly well in practice, especially for text classification with high-dimensional sparse features.

Gaussian Mixture Models

A GMM is a soft clustering model. It models the data as a mixture of K Gaussian distributions, each with its own mean and covariance. Unlike k-means (hard assignment), GMM assigns every point a probability of belonging to each cluster. Fitting uses the Expectation-Maximisation (EM) algorithm — itself a probabilistic inference method.

Left: probabilistic graphical model of Naive Bayes. All features are conditionally independent given the class node. Right: a three-component GMM. Each Gaussian (dashed) is a soft cluster. The mixture (yellow) can model complex multi-modal distributions.

ML Connection — Latent Variable Models

GMMs are the simplest example of a latent variable model: the cluster assignment is latent (unobserved). VAEs are deep generative GMMs — the latent variable is continuous, the encoder approximates the posterior over it, and the decoder is the likelihood. Diffusion models add Gaussian noise progressively (forward process) and learn to reverse it — again using Gaussian distributions and their probabilistic properties throughout.

GMM: $P(x) = \Sigma k\ \pi k \cdot N(x; \mu k, \Sigma k)$. EM: E-step computes $P(z|x, \theta_old)$; M-step maximises expected log-likelihood. BIC/AIC criteria select number of components K.

Part VII · Chapter 17

Evaluation Metrics from a Probabilistic Lens

Accuracy is not enough. Classification thresholds, class imbalance, uncertainty calibration — all require richer evaluation. Confusion matrices, ROC curves, AUC, precision-recall, and calibration plots all have direct probabilistic interpretations.

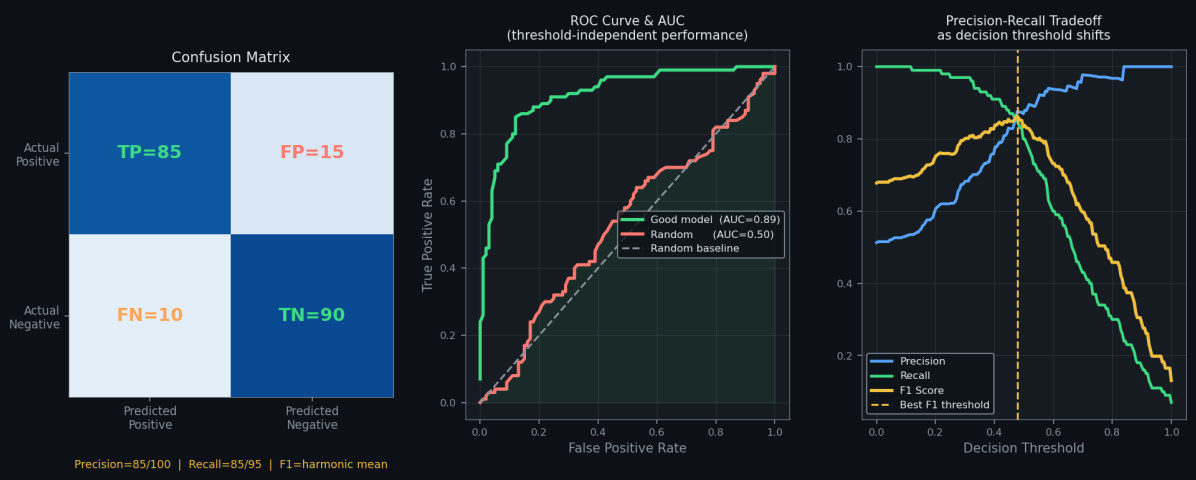
The Confusion Matrix — Ground Truth of Classification

Every binary classification model produces a confusion matrix: True Positive, False Positive, False Negative, True Negative. All other metrics are derived from these four numbers.

- **Precision** = TP/(TP+FP): of all predicted positives, how many are actually positive?
- **Recall (Sensitivity)** = TP/(TP+FN): of all actual positives, how many did you find?
- **F1 Score** = harmonic mean of precision and recall — good when classes are imbalanced.
- **Specificity** = TN/(TN+FP): true negative rate — how well does the model avoid false alarms?

ROC and AUC — Threshold-Independent Evaluation

ROC (Receiver Operating Characteristic) plots TPR vs FPR as the decision threshold varies. AUC (Area Under the Curve) summarises the whole curve as one number. AUC = probability that a random positive is ranked higher than a random negative. A random classifier has AUC=0.5. A perfect one has AUC=1.0.



Left: confusion matrix with standard colour coding (green=correct, red=errors). Middle: ROC curves for a good model and a random baseline. The AUC is the probability of correct ranking. Right: precision-recall tradeoff as the threshold changes.

ML Connection — Calibration

A well-calibrated classifier means: when it says P(positive)=0.7, 70% of those predictions are actually positive. Reliability diagrams (calibration curves) visualise this. Temperature scaling calibrates a model's softmax output post-hoc by dividing logits by a temperature $T > 1$ to soften overconfident predictions. Expected Calibration Error (ECE) measures miscalibration numerically — critical for medical, legal, and financial ML applications.

Precision-Recall AUC better than ROC-AUC for imbalanced datasets. Matthews Correlation Coefficient (MCC) is a single number summarising the full confusion matrix. Log loss = average negative log-likelihood of predictions (penalises confidence on wrong predictions).

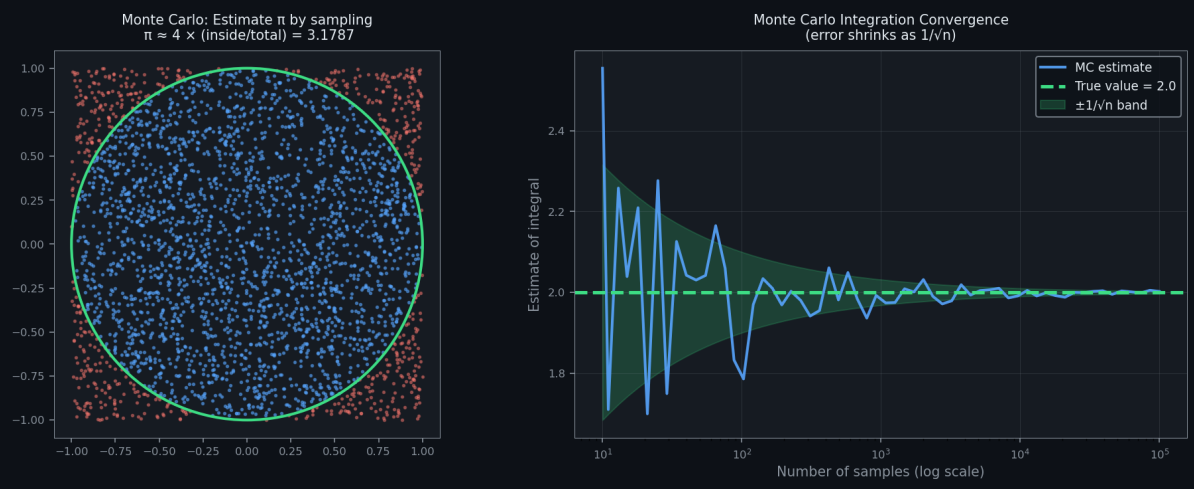
Part VII · Chapter 18

Monte Carlo Methods

Monte Carlo methods solve intractable problems by sampling. When you cannot compute an integral analytically, you approximate it by averaging over random samples. This powers Bayesian inference, dropout at test time, reinforcement learning policy evaluation, and generative modelling.

Monte Carlo Integration

To compute E[f(X)] for some complex distribution, draw N samples x1,...,xN from that distribution, compute f(xi) for each, and average. The error of this estimate shrinks as 1/sqrt(N) — regardless of dimensionality. This is why Monte Carlo scales to high dimensions where numerical integration fails.



Left: estimating pi by sampling random points and checking if they fall inside the unit circle. Right: Monte Carlo integral convergence. The estimate fluctuates early then settles to the true value, with uncertainty shrinking as 1/sqrt(n) (the green band).

Markov Chain Monte Carlo (MCMC)

When you cannot even sample from the target distribution directly, MCMC constructs a Markov chain whose stationary distribution equals the target. You run the chain for many steps and collect samples — they will eventually represent the target distribution. Metropolis-Hastings and Hamiltonian Monte Carlo (HMC) are the most important algorithms.

ML Connection — Monte Carlo in Deep Learning

MC Dropout: run inference multiple times with dropout enabled. The variance of predictions approximates Bayesian uncertainty. Monte Carlo Tree Search (MCTS): used in AlphaGo and AlphaZero to evaluate game states by sampling many random rollouts. Reinforcement Learning policy gradient (REINFORCE) estimates the gradient of expected reward by sampling trajectories — pure Monte Carlo. Importance Sampling: reweight samples from a tractable distribution to estimate expectations under an intractable one. Used in off-policy RL and normalising flows.

MC estimate: (1/N)·Σf(x■) → E[f(X)] as N→∞ (Law of Large Numbers). Variance of estimate = Var[f(X)]/N. Importance sampling: E_p[f] ≈ (1/N)·Σ f(x■)·p(x■)/q(x■) where x■ ~ q.

Appendix A

Quick Reference Formula Sheet

Every key formula from this document, condensed onto one visual page. Use this as a daily reference while working through ML theory or reading papers.

Quick Reference: Key Rules

Bayes:	$P(H\|D) = P(D\|H)\cdot P(H) / P(D)$
Total Prob.:	$P(A) = \Sigma P(A\|B_i)\cdot P(B_i)$
Expectation:	$E[X] = \Sigma x\cdot P(X=x)$ or $\int x\cdot f(x)dx$
Variance:	$Var[X] = E[X^2] - (E[X])^2$
Std. Deviation:	$\sigma = \sqrt{Var[X]}$ (same units as X)
Covariance:	$Cov[X,Y] = E[(X-\mu x)(Y-\mu y)]$
Correlation:	$\rho = Cov[X,Y] / (\sigma x\cdot\sigma y) \in [-1,1]$
Entropy:	$H(X) = -\Sigma P(x)\cdot\log P(x)$
Cross-Entropy:	$CE(P,Q) = -\Sigma P(x)\cdot\log Q(x)$
KL Divergence:	$KL(P\|\|Q) = \Sigma P(x)\cdot\log(P(x)/Q(x))$
MLE:	$\theta^* = \arg\max \Sigma \log P(x_i\|\theta)$
MAP:	$\theta^* = \arg\max [\Sigma \log P(x_i\|\theta) + \log P(\theta)]$
Normal PDF:	$f(x) = (1/\sigma\sqrt{2\pi})\cdot\exp(-(x-\mu)^2/2\sigma^2)$
CLT:	$\bar{X} \sim N(\mu, \sigma^2/n)$ as $n \to \infty$

All essential probability and statistics formulas. Colour-coded by topic: blue = core probability, green = expectation/variance, purple = covariance, orange = information theory, red = estimation, cyan = distributions and CLT.

Distribution Quick Reference

Distribution	Support	Mean	Variance	ML Use Case
Bernoulli(p)	{0,1}	p	p(1-p)	Binary output, Dropout
Binomial(n,p)	{0..n}	np	np(1-p)	Accuracy over n tests
Poisson(λ)	{0,1,2...}	λ	λ	Token counts, anomalies
Gaussian(μ,σ²)	R	μ	σ²	Weights, noise, latents
Beta(α,β)	[0,1]	α/(α+β)	αβ/((α+β)²(α+β+1))	CTR prior, Beta-VAE
Exponential(λ)	[0,∞)	1/λ	1/λ²	Waiting times, survival
Uniform(a,b)	[a,b]	(a+b)/2	(b-a)²/12	Random init, sampling
Dirichlet(α)	Simplex	αi/Σαi	see formula	Topic models, LDA
Categorical(p)	{1..K}	—	—	Softmax output class

The Big Picture

Every component of a modern ML system is probabilistic at its core. The model outputs a distribution, not a point. The loss function is a negative log-likelihood. The optimiser is following noisy gradient estimates. The evaluation metrics are statistical estimators with confidence intervals. The regulariser is a prior. The architecture encodes inductive biases about structure.

Probability is not a topic you study and move on from. It is the lens through which every ML decision becomes interpretable. Why does this loss function make sense? Probability. Why is this model overconfident? Probability. Why does this ensemble work? Probability. The more fluently you speak it, the more clearly you see what your models are actually doing.

The goal was never to memorise formulas. The goal is to see that training a neural network is Bayesian inference, gradient descent is climbing a probability landscape, and every prediction your model makes is a statement about uncertainty — whether the model knows it or not.